

Cooperative Kernel Regression

Distributed Optimization, Federated Learning, and Differential Privacy

Agathe PASCAL
Nastassia BONETTI

March 29, 2026

Abstract

This report presents a course-consistent implementation of Gaussian-kernel ridge regression with a Nyström approximation. The experiments cover distributed optimization over a communication graph, federated learning, and a differentially private distributed variant. The focus is on the mathematical formulation, stable parameter choices, convergence plots, and a concise interpretation of the observed behavior of each method.

1 Common formulation

The kernel is

$$k(x, \xi) = \exp(-(x - \xi)^2).$$

Given n samples (x_i, y_i) and m Nyström landmarks $\tilde{x}_1, \dots, \tilde{x}_m$, define

$$(K_{mm})_{pq} = k(\tilde{x}_p, \tilde{x}_q), \quad (K_{nm})_{ip} = k(x_i, \tilde{x}_p).$$

The centralized problem is

$$\alpha^* \in \arg \min_{\alpha \in \mathbb{R}^m} \frac{\sigma^2}{2} \alpha^\top K_{mm} \alpha + \frac{1}{2} \|y - K_{nm} \alpha\|^2 + \frac{\nu}{2} \|\alpha\|^2,$$

with $\sigma = 0.5$ and $\nu = 1$. Since the objective is quadratic and strongly convex,

$$\alpha^* = \left(K_{nm}^\top K_{nm} + \sigma^2 K_{mm} + \nu I_m \right)^{-1} K_{nm}^\top y.$$

This solution is used as the reference point throughout the experiments.

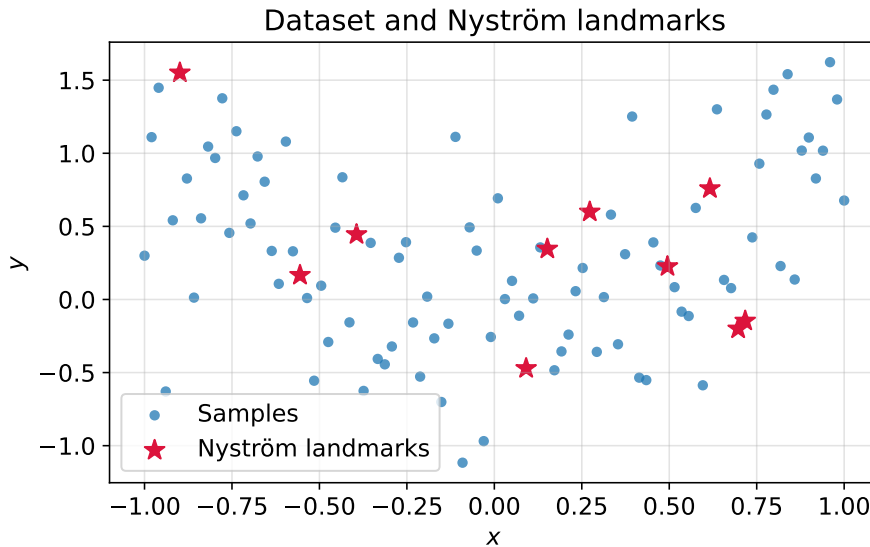


Figure 1: Samples used in Parts I and III, together with the selected $m = 10$ Nyström landmarks.

For the distributed setting, the first $n = 100$ points are split over $a = 5$ agents, hence 20 points per agent. With local data (K_i, y_i) at agent i ,

$$f_i(\alpha) = \frac{\sigma^2}{2a} \alpha^\top K_{mm} \alpha + \frac{1}{2} \|y_i - K_i \alpha\|^2 + \frac{\nu}{2a} \|\alpha\|^2, \quad \sum_{i=1}^a f_i(\alpha) = F(\alpha).$$

We monitor

$$g_i^t = \|\alpha_i^t - \alpha^\star\|, \quad \bar{g}^t = \|\bar{\alpha}^t - \alpha^\star\|, \quad \bar{\alpha}^t = \frac{1}{a} \sum_{i=1}^a \alpha_i^t.$$

2 Distributed optimization

Four graphs are tested: cycle, line, small-world, and complete. The undirected mixing matrices are built with Metropolis weights, so the standard course assumptions hold: symmetry, stochasticity, and connectivity.

The methods are DGD, Gradient Tracking (GT), dual decomposition, and ADMM. Their parameters are chosen from the local objective constants

$$L_{\max} = \max_i \lambda_{\max}(H_i), \quad \mu_{\min} = \min_i \lambda_{\min}(H_i),$$

the graph mixing factor $\beta = \|W - J\|_2$, and for the dual method

$$L_d = \|B^\top H^{-1} B\|_2.$$

- **DGD:** $\eta_{\text{DGD}} = \min\left(0.9 \frac{1-\beta}{L_{\max}}, 0.9 \frac{1}{L_{\max}}\right)$. This keeps the method in the stable $O((1-\beta)/L)$ regime. With a constant step size, DGD converges linearly to a neighborhood of $\alpha^\star$ because the consensus error does not vanish exactly.
- **Gradient Tracking:** $\eta_{\text{GT}} = 0.2/L_{\max}$. The tracking variable reconstructs the global average gradient; under smoothness and strong convexity, a small enough $O(1/L)$ step yields exact linear convergence.
- **Dual decomposition:** $\tau_{\text{dual}} = 1/(2L_d)$. The dual objective is smooth and concave, so this is a conservative stable ascent step that leads here to the exact solution.
- **ADMM:** $\rho_{\text{ADMM}} = \sqrt{\mu_{\min} L_{\max}}$. In quadratic consensus problems, any $\rho > 0$ gives the exact solution; this choice balances the primal and dual scales.

Part I - line graph

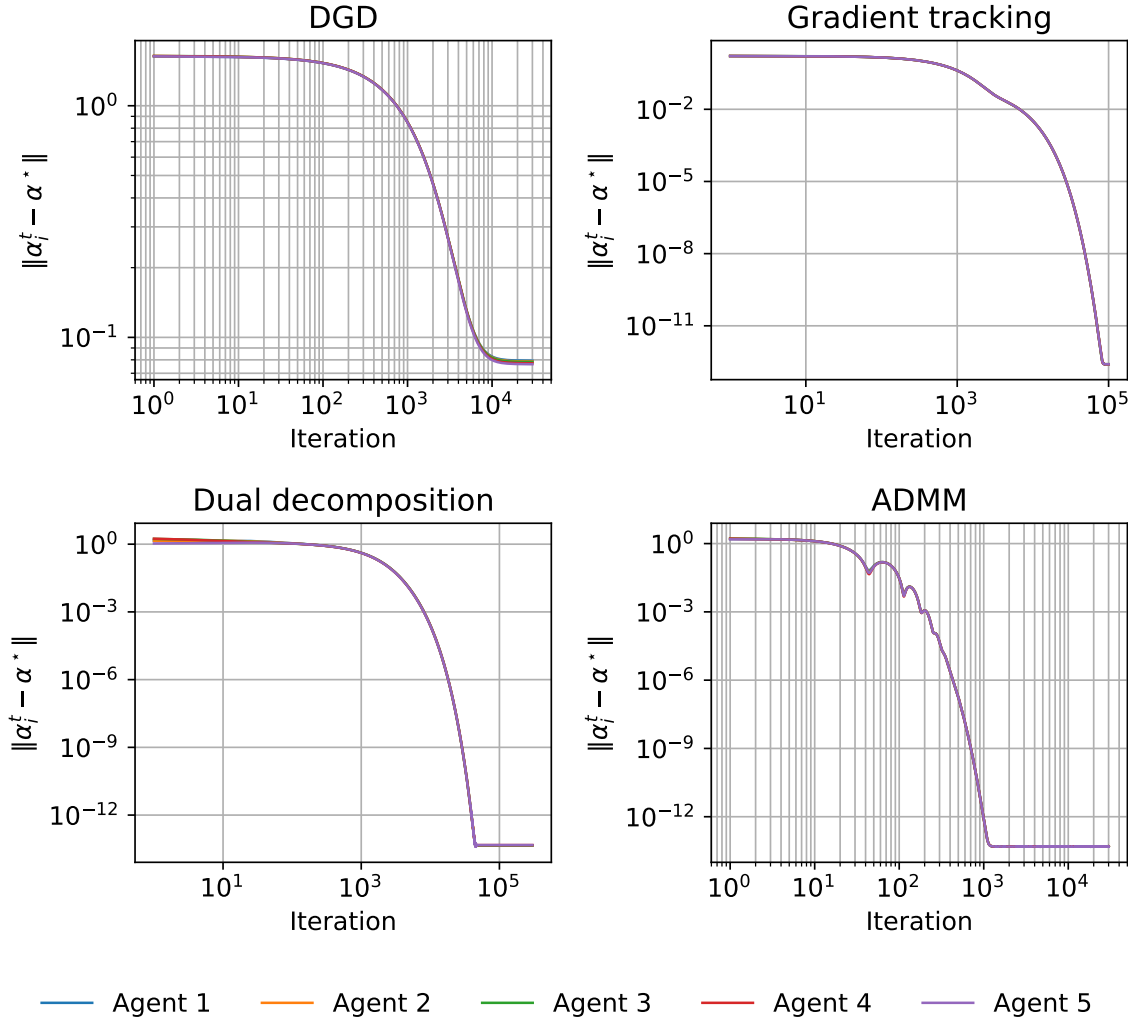


Figure 2: On the line graph, DGD converges to a neighborhood of the solution, whereas GT, dual decomposition, and ADMM reach numerical precision.

The line graph is representative: the worst final DGD gap is 7.93×10^{-2} , while the three other methods end below 4×10^{-13} . This matches the theory exactly: DGD with a constant step retains a consensus bias, whereas GT, the dual method, and ADMM are exact methods in this quadratic setting.

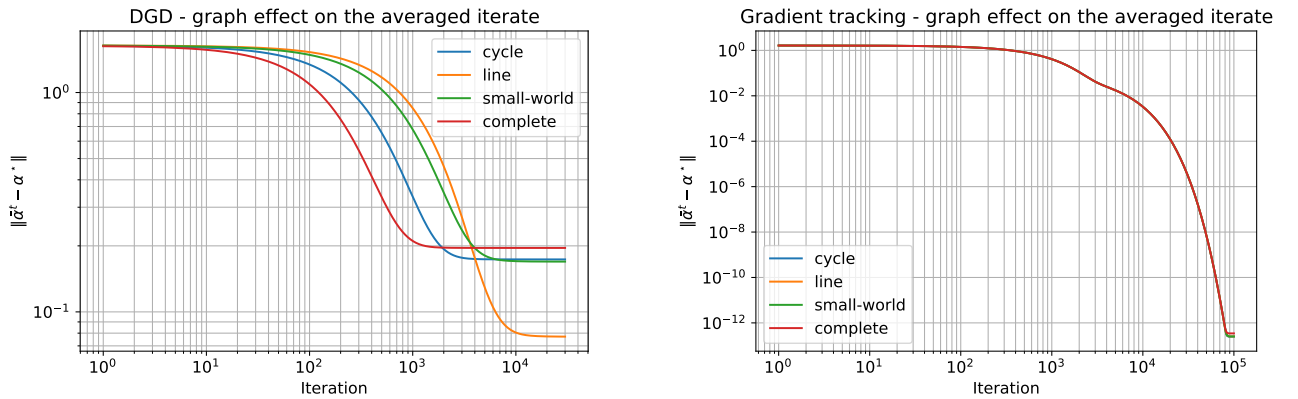


Figure 3: Influence of the graph topology on the averaged iterate for DGD (left) and GT (right).

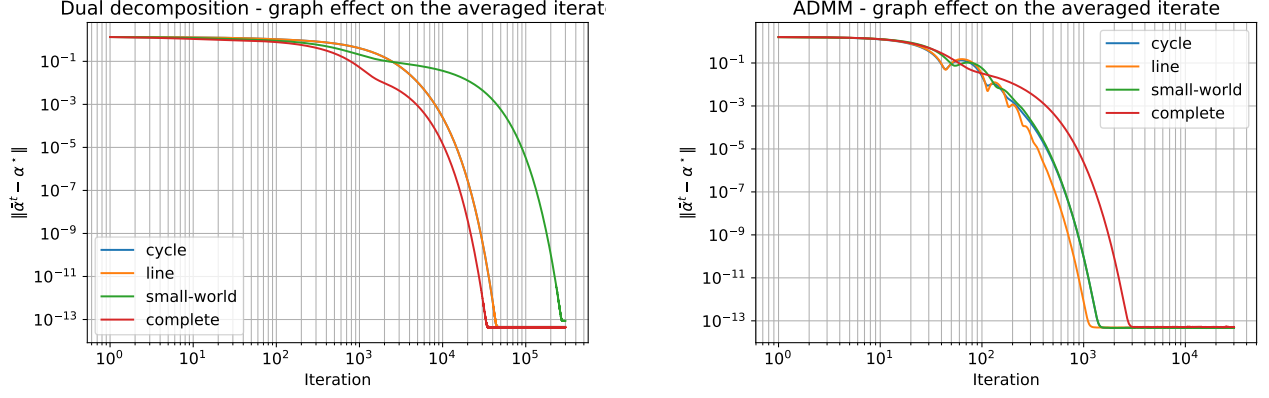


Figure 4: Influence of the graph topology on the averaged iterate for Dual decomposition (left) and ADMM (right).

Topology affects the methods differently. For DGD, better mixing mostly improves the transient phase and allows a larger admissible step; because $\eta_{\text{DGD}} \propto (1 - \beta)/L_{\max}$, the final bias is therefore not strictly monotone in β . For GT, the curves align much faster and all end at machine precision, confirming that the tracking term removes the steady-state bias observed in DGD. Dual decomposition and ADMM also reach the exact solution here because the consensus problem is quadratic, strongly convex, and solved through closed-form local updates.

We can also take a look at the function reconstruction for each algorithm.

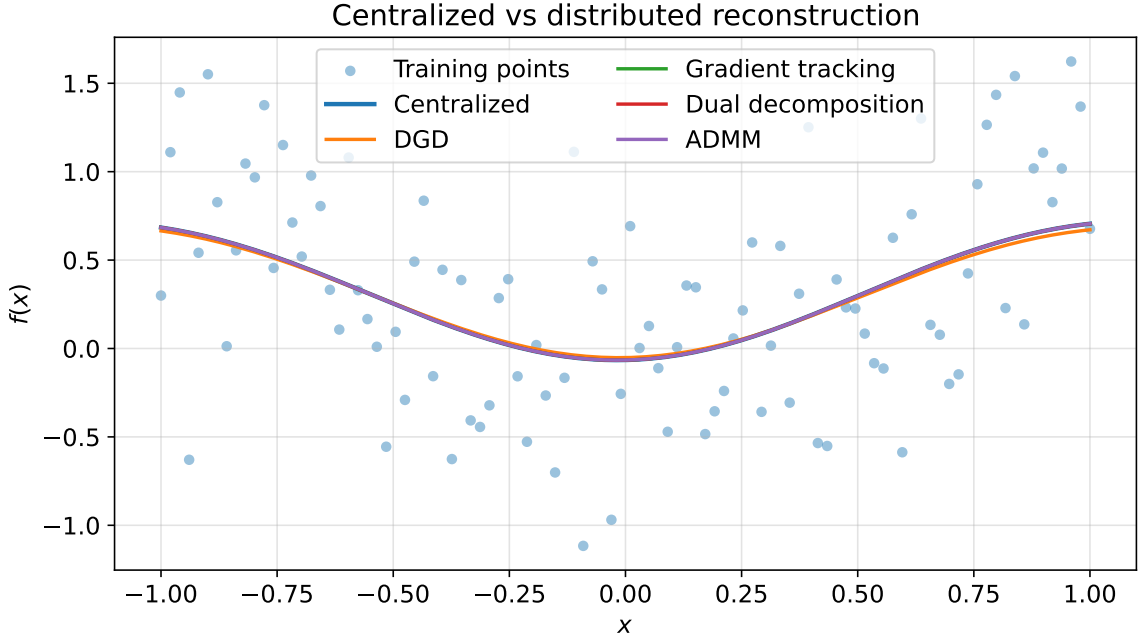


Figure 5: Visualisation of the functions for each algorithms.

We can see that for **DGD**, the curve is slightly offset from the others. This may be due to its bias, since a constant step size was used here. There is still an error. If we look at **GT**, **ADMM**, and **DC**, the three curves are virtually indistinguishable.

We can try changing several parameters on the communication graphs to see how they affect the algorithms and their convergence: Directed, Packet loss, Asynchronous. For now, if we make these changes, we obtain the following results for the DGD:

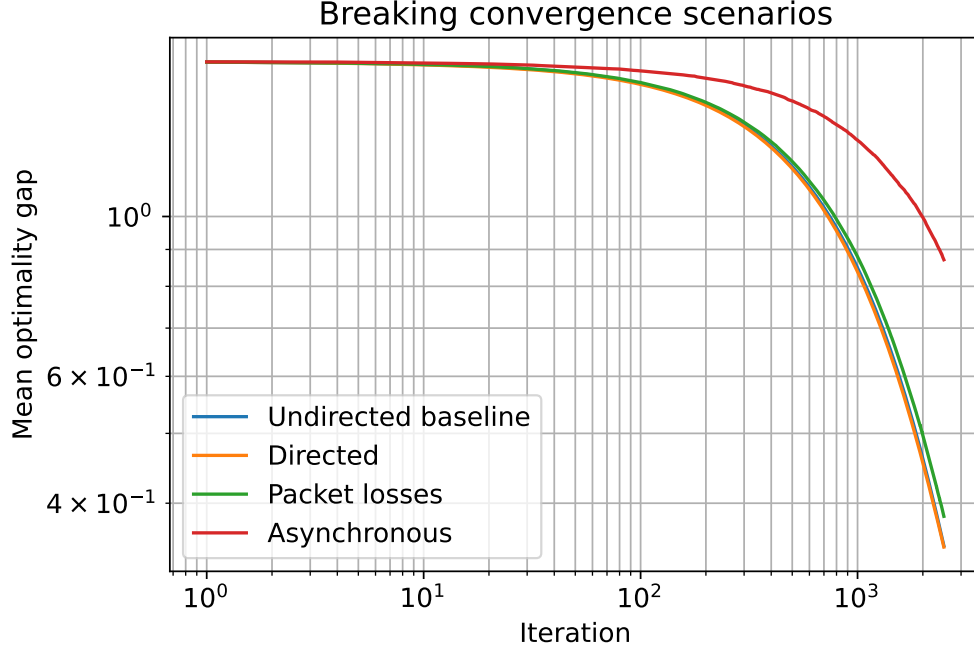


Figure 6: Convergence broken

Because of these modifications, we can use the Push-sum algorithm to solve the convergence problem. We have tried to implement it on DGD. Here are the results. This method helps DGD to have a lower error and converge faster.

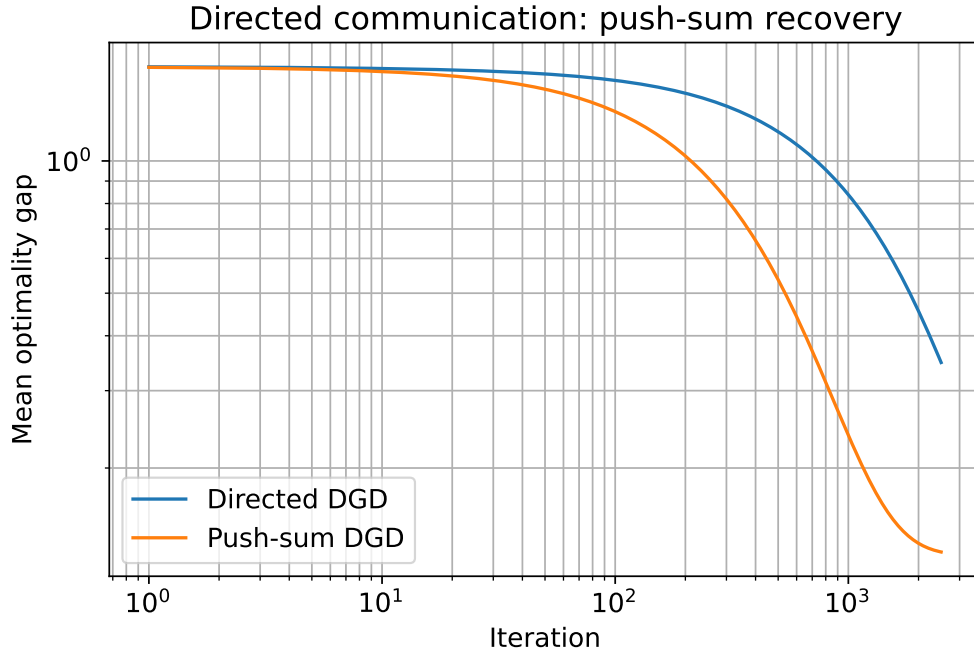


Figure 7: Convergence broken

Finally, we can study the case of a larger n . In order to maintain a valid Nyström approximation, we increase the model dimension m in proportion to $\sqrt{n}$.

We have implemented a dynamic search procedure to determine the system's maximum capacity. The algorithm iteratively increases n and measures two key metrics:

- **Final accuracy:** The optimality gap reached after a fixed number of iterations.
- **Convergence rate:** The number of iterations required to fall below a critical threshold.

Examining the generated figures (speed vs. n) allows us to draw several conclusions:

1. **Computational Complexity:** Although the problem is distributed among a agents, an increase in n (and thus in m) increases the cost of each local operation.
2. **Consensus cost:** As n increases, the structure of the problem becomes more complex, and the number of iterations required to reach consensus increases.

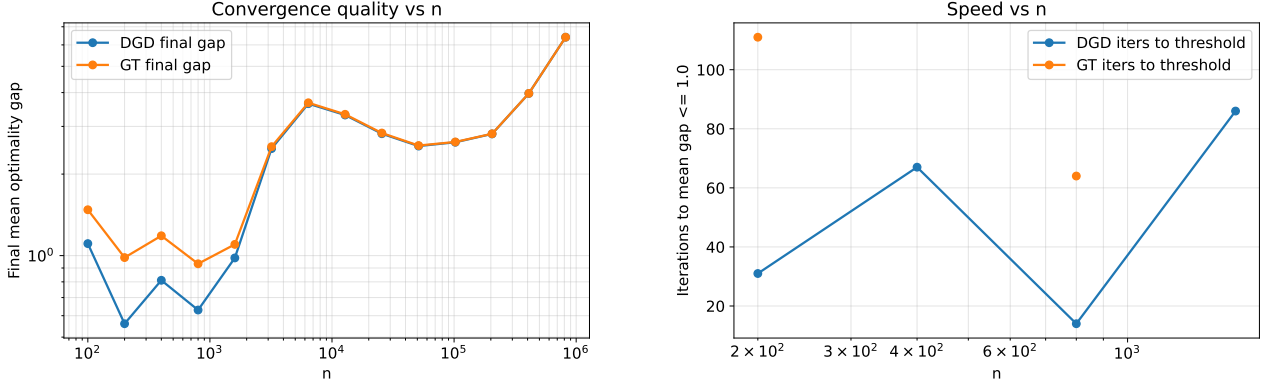


Figure 8: Speed and optimality ga

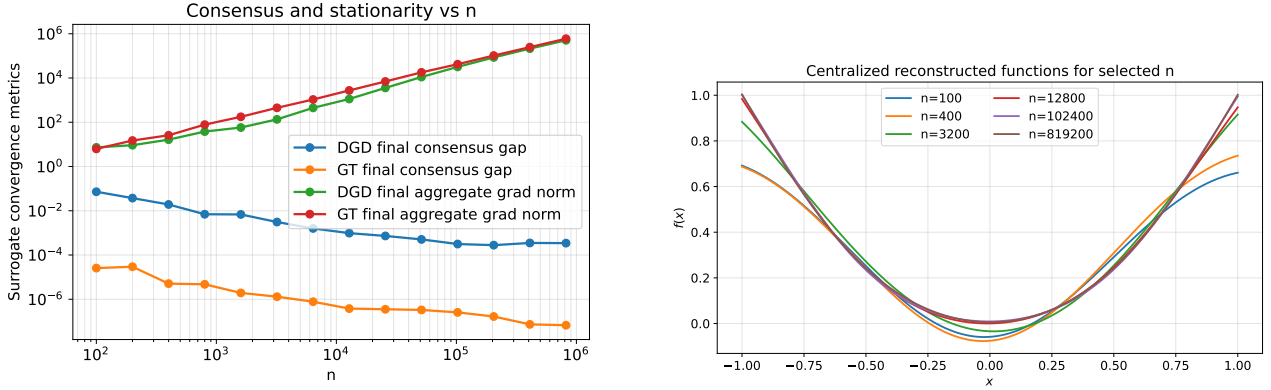


Figure 9: Functions visualisation

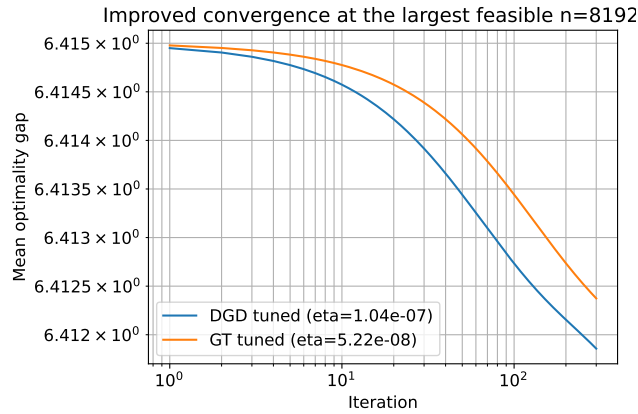


Figure 10: The maximum found is **8192**

3 Federated learning

The second dataset contains five clients with 20 points each. The global objective keeps the same Nyström structure:

$$F(\alpha) = \sum_{c=1}^5 \frac{1}{2} \|y_c - K_c \alpha\|^2 + \frac{\sigma^2}{2} \alpha^\top K_{mm} \alpha + \frac{\nu}{2} \|\alpha\|^2.$$

The local mini-batch gradient used by FedAvg and SCAFFOLD is unbiased:

$$g_{c,b}(\alpha) = \frac{n_c}{|b|} K_{c,b}^\top (K_{c,b} \alpha - y_{c,b}) + \frac{\sigma^2}{5} K_{mm} \alpha + \frac{\nu}{5} \alpha.$$

The baseline local step size is

$$\eta_{\text{FedAvg}} = \frac{0.25}{L_{\max}^{\text{loc}}} = 3.136 \times 10^{-3}.$$

This keeps the local updates in a stable $O(1/L)$ regime. When $E = 50$, the step is halved to limit client drift, which naturally grows with the number of local epochs between two aggregations.

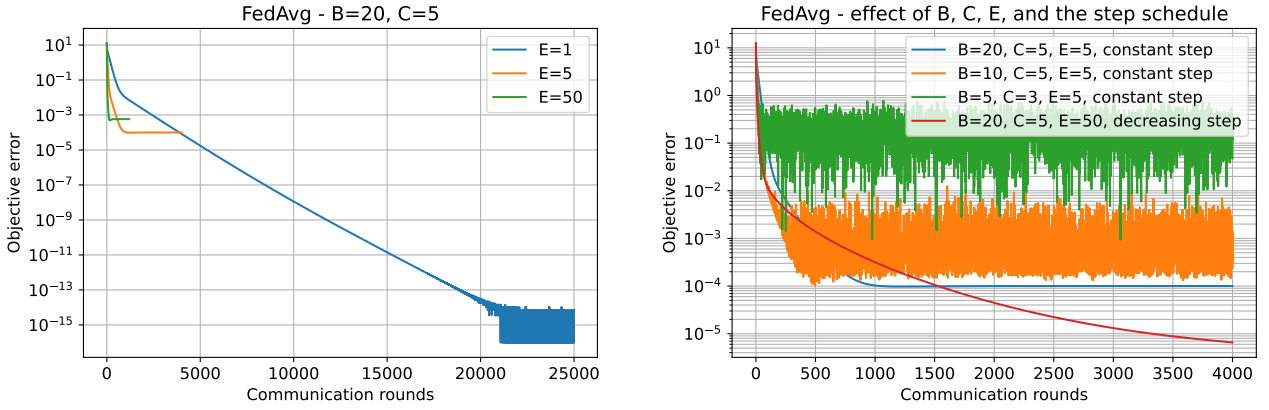


Figure 11: FedAvg with $B = 20, C = 5, E \in \{1, 5, 50\}$ (left), and the effect of B, C, E , and the step schedule (right).

The results display the usual trade-off. For $E = 50$, descent per communication round is fast, but it requires a more cautious step size. For $E = 1$, the trajectory is much slower in communication rounds, yet it also reaches machine precision when the horizon is long enough. In the displayed runs, the final objective errors are $< 10^{-15}$ for $E = 1$, 1.00×10^{-4} for $E = 5$, and 5.81×10^{-4} for $E = 50$. The sensitivity plot confirms that decreasing B increases local variance, decreasing C slows down the global correction, and a decreasing step is mainly useful in the most aggressive settings.

The optional comparison with SCAFFOLD, run with $C = 3$ and $E = 5$, gives a final error 2.03×10^{-2} for FedAvg versus 2.13×10^{-14} for SCAFFOLD. This is consistent with the theory: the control variates compensate for client drift and become especially useful as soon as not all clients participate at every round.

4 Differential privacy

To solve the regression problem in a distributed manner while ensuring local data privacy, we implement the DGD-DP algorithm. Following the framework of **Theorem 2** from [?], the private update rule is defined as follows:

$$\alpha^{k+1} = \alpha^k + \gamma_k (W(\alpha^k + \zeta^k) - \alpha^k) - \eta_k g^k$$

where g_i^k is the local gradient and ζ_i^k represents the additive Laplace noise injected during the communication step.

Noise Mechanism and Sensitivity To satisfy ϵ -differential privacy requirements, the noise is drawn from a Laplace distribution $\zeta_i^k \sim \text{Lap}(0, \nu)$, where the scale parameter ν is calibrated based on the local sensitivity C and the privacy budget ϵ :

$$\nu = \frac{C}{\epsilon}$$

In this implementation, rather than using explicit clipping, we assume the local gradient sensitivity is naturally bounded by $C = 0.01$. This choice of ν ensures that the privacy budget is respected at each exchange, protecting the local model parameters from being reconstructed by neighboring agents.

Convergence Conditions The persistent injection of noise necessitates the use of decaying step-size sequences to ensure stability and convergence toward the optimal solution α^* . We employ the following polynomial decay schedules:

$$\gamma_k = \frac{1}{1 + 0.001 \cdot k^{0.9}}, \quad \eta_k = \frac{0.002}{1 + 0.001 \cdot k}$$

- $\epsilon = 10$: Low noise level; the scale ν is small, allowing for fast convergence that closely matches the deterministic DGD behavior.
- $\epsilon = 0.1$: High privacy level; the large noise scale ν induces significant variance in the consensus step. However, the decaying steps η_k and γ_k successfully dampen these oscillations, allowing the algorithm to stabilize in a neighborhood of the optimum.

Part III - private DGD

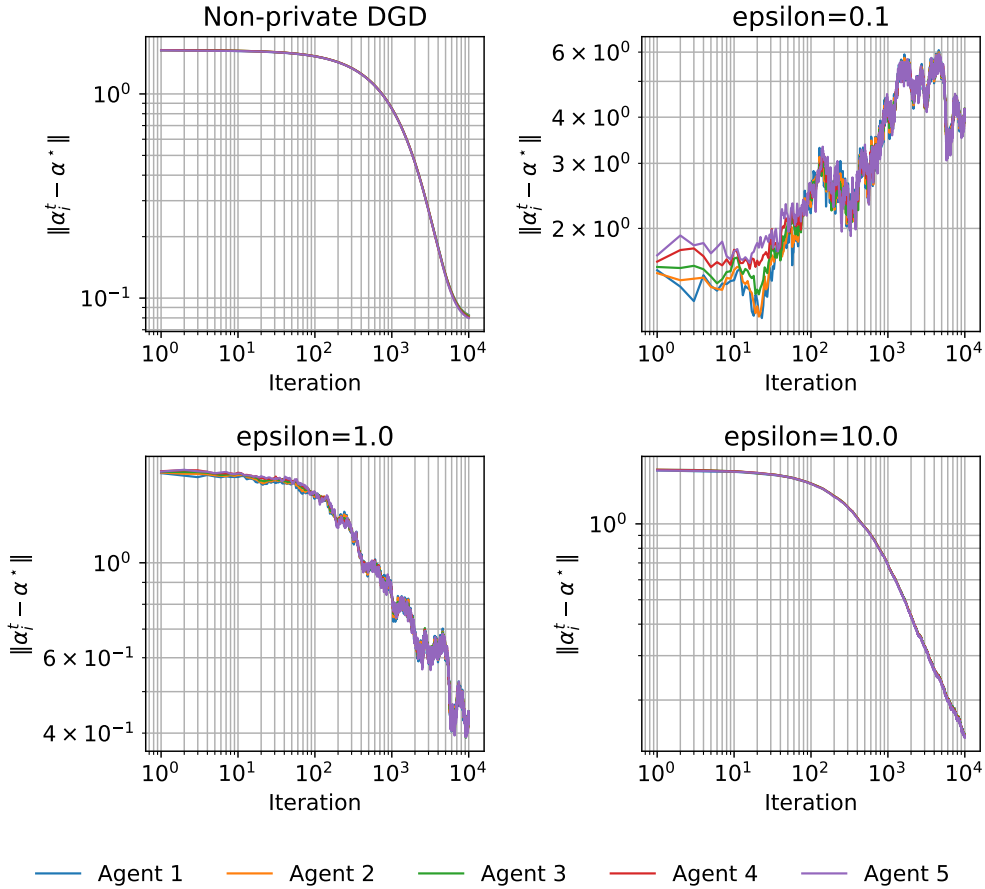


Figure 12: Agent-wise gaps for non-private DGD and private DGD with $\epsilon \in \{0.1, 1, 10\}$.

The analysis in Figure 12 highlights the fundamental trade-off in *Differential Privacy*.

- **Strong privacy** ($\epsilon = 0.1$): A high error plateau is observed. The injected Laplace noise dominates the gradient signal in the early iterations, preventing the algorithm from achieving high precision.
- **Moderate privacy** ($\epsilon = 10$): The curve converges to an error level close to that of classical DGD. The noise is sufficiently low to be absorbed by the gradient descent dynamics.
- **Effect of decreasing steps**: Unlike standard DGD, the use of decreasing steps η_t and γ_t is vital here. It helps stabilize noise-induced oscillations, although it slows down the overall convergence rate.

5 Conclusion

The experiments support the following conclusions:

- the Nyström kernel-ridge formulation is used consistently across the three parts, with the centralized solution as the reference point;
- in distributed optimization, DGD keeps a constant-step bias, whereas GT, dual decomposition, and ADMM converge exactly in this quadratic setting;
- in federated learning, increasing the number of local epochs speeds up progress per round but also increases client drift, and SCAFFOLD corrects that drift effectively;
- in differential privacy, DGD-DP remains stable but converges to a neighborhood, as expected with Gaussian noise and a constant step.

The numerical behavior is therefore coherent with the course notation and with the expected mechanisms of the different algorithms.